

Finding Transformer Circuits with Edge Pruning

Authors: Adithya Bhaskar, Alexander Wettig, Dan Friedman, Danqi Chen

Year: 2024 · Venue: arXiv (Cornell University) · Difficulty: ●●●○○ (Advanced)

[10.48550/arxiv.2406.16778](https://arxiv.org/abs/2406.16778)

TL;DR

The paper turns circuit discovery in Transformers into a direct optimization problem: instead of greedily testing edges or using rough gradient scores, it learns which connections to keep with trainable edge masks, producing much sparser and more faithful circuits and scaling all the way to CodeLlama-13B.

Prerequisites

- Transformer (a neural network for sequences where attention layers and MLPs repeatedly read from and write to a shared running state called the residual stream)
- Residual stream (the running hidden state in a Transformer that each layer adds to, like many workers writing notes into the same shared notebook)
- Attention head (one small attention module that decides which earlier tokens matter and passes along a weighted summary)
- Circuit in mechanistic interpretability (a sparse subgraph of model components and connections that is supposed to explain one behavior of the full model)
- Pruning (removing parts of a model or computation graph to keep only what seems necessary)
- Gradient descent (the standard method for improving parameters by following derivatives of a loss function)
- KL divergence (a way to measure how different two probability distributions are, often used to compare model output distributions)
- Activation patching / interchange ablation (replacing an internal activation with one from a different input to test whether that activation matters causally)
- L0 regularization (a way to encourage exact sparsity by penalizing the number of nonzero choices, i.e. how many things you keep)

The Big Idea

This paper is about finding “circuits” in language models automatically. A circuit is the small set of internal connections that actually matters for one behavior, like solving an indirect object identification task or following a prompt format. The dream is simple: instead of staring at a giant Transformer with millions of moving parts, we want a tiny wiring diagram that still behaves like the full model on the task we care about. That is the interpretability payoff. If you can shrink the computation without breaking it, you have a much better shot at understanding what the model is doing.

The status quo before this paper was unsatisfying in two different ways. One line of work, like ACDC, is careful but slow: it removes edges one at a time and checks what breaks. That is like debugging a machine by unscrewing one wire at a time. It works, but it does not scale. Another line, like EAP, is fast but approximate: it uses gradients to guess which edges matter, without actually solving the pruning

problem. That is like looking at a map of slopes and guessing where the important roads are, instead of driving them. The authors’ pitch is that we do not need to choose between slow search and crude approximation. We can directly optimize for a sparse circuit.

Their core move is clever and pretty clean. They rewrite the Transformer so the residual stream is “disentangled” (meaning: instead of one big mixed vector, they keep track of which earlier component contributed what). Once you do that, you can put a learnable mask on each edge. Each mask says, in effect, “keep this connection” or “replace it with a counterfactual value from a corrupted example.” Then you optimize those masks with gradient descent while pushing for sparsity. In plain English: the method learns which wires can be cut while still making the reduced model match the full model.

Why should you care? Because this is one of the first circuit-finding methods that looks genuinely practical at larger scale. On GPT-2 tasks, it finds smaller circuits than prior methods without losing faithfulness. On Tracr models (toy Transformers compiled from known programs), it recovers the true circuit exactly. And most importantly, it scales to CodeLlama-13B, where earlier automated methods basically hit a wall. That scaling result matters. A lot of interesting behaviors only show up in bigger models, so a circuit method that only works on tiny models is not enough.

The Problem

The formal problem is: given a full model graph G , find a sparse subgraph $C \subseteq G$ that still reproduces the full model’s behavior on a task. The paper writes this as minimizing a discrepancy between the full model output and the circuit output, usually with KL divergence (a distance between output distributions):

$$\arg \min_C \mathbb{E}_{(x, \tilde{x}) \in T} [D(p_G(y | x) \parallel p_C(y | x, \tilde{x}))]$$

subject to a sparsity target like

$$1 - |C|/|G| \geq c.$$

That is the whole game: match the original model while keeping very few edges. The hard part is that this is a nasty combinatorial problem (meaning: there are exponentially many possible subgraphs, so brute force is hopeless).

There is also a subtle causal issue. If you simply delete an edge, the downstream activations may become unrealistic, like removing a gear from a watch and then acting surprised that the rest of the mechanism is now in a weird state. So the paper follows the “interchange ablation” idea: when an edge is removed, its contribution is replaced by the corresponding activation from a corrupted input $\tilde{x}$. That gives a cleaner counterfactual: “what if this piece of information came from the wrong example?” rather than “what if this signal became zero?” This is important because zeroing things out often creates junk states the model was never trained to handle.

Previous methods each miss something important. ACDC does greedy search over edges. That means it repeatedly asks, “if I remove this one edge right now, how bad is it?” The problem is that edge importance is highly interactive. Two edges may each look useless alone but be essential together, or vice versa. Greedy search also scales terribly because you keep re-running expensive interventions. EAP is the opposite tradeoff: it uses a first-order approximation (a linear local estimate from gradients) to score all edges at once. Fast, yes. But local linear guesses are often wrong for these highly nonlinear systems. The authors’ main motivation is that circuit discovery should be treated as an optimization problem over the whole edge set, not as either a one-edge-at-a-time search or a one-shot approximation.

Key Contributions

-
- They propose Edge Pruning, a method that learns binary-like masks on edges between Transformer components rather than pruning whole heads or layers. This matters because circuits are really about connections, not just components, and edge-level control is much more precise.
 - They adapt the Transformer into a disentangled residual-stream form so individual upstream contributions can be selectively read or replaced. That engineering step is what makes edge-level pruning possible at all; without it, the residual stream is too mixed together to cleanly cut specific connections.
 - They optimize circuit discovery directly with gradient-based sparsification, using L0-style regularization (a penalty that pushes the number of kept edges down). This is the conceptual heart of the paper: instead of approximating the search problem, they solve a relaxed version of it end-to-end.
 - They show better empirical results than prior automated circuit-finding methods on standard GPT-2 tasks, especially on harder settings like multi-template IOI. The headline is not just smaller circuits, but smaller circuits that are still faithful to the full model's predictions.
 - They perfectly recover ground-truth circuits in Tracr models, which are compiled from known programs. That is a strong sanity check: on systems where we know the right answer, the method gets the right answer.
 - They scale automated circuit discovery to CodeLlama-13B and find circuits with about 99.96% sparsity that still match full-model performance on a Boolean-expression task. This is the paper's biggest practical win, because it pushes circuit discovery into the regime where interesting large-model behaviors live.
 - They use that scale to compare instruction prompting and in-context learning, and find that the two resulting circuits overlap a lot. That is scientifically interesting: it suggests these two prompting modes are not totally different mechanisms, at least for this task.

Where This Fits

This paper sits in the mechanistic interpretability line of work that tries to explain model behavior by identifying circuits: sparse sets of components and connections that implement a function. Earlier circuit work was mostly manual, with researchers tracing attention heads and MLPs by hand. Then came automated methods like ACDC, which used activation patching to prune edges by greedy search, and EAP, which used attribution-style gradient scores to rank edges. Edge Pruning is very much in conversation with those papers. It keeps the same basic goal and similar evaluation setup, but changes the optimization strategy from search or approximation to direct sparse learning.

It also borrows from the pruning literature, but in a different spirit than model compression. Standard pruning usually asks, “how can we remove parameters or channels while keeping accuracy and maybe speeding up inference?” Here the goal is not speed. It is interpretability. That changes what should be pruned and how. The authors argue, correctly in my view, that pruning nodes is too coarse for circuit discovery. The meaningful object is often the edge: which earlier computation is being read by which later one. So the paper is really a hybrid of two traditions: mechanistic interpretability gives the target object, and sparse optimization gives the tool. That combination is the main reason the paper feels like a real step forward rather than just another scoring heuristic.

How It Works

First, the simple version: they turn circuit finding into a pruning problem. But instead of asking "which heads or MLPs matter?" they ask the finer question: "which specific connections matter?" Think of a Transformer as a city map. Older pruning methods close whole buildings. Edge Pruning closes individual roads between buildings. That is much more useful if you want to understand the exact route information takes.

Concretely, they freeze the original Transformer weights and add one trainable mask parameter $z_{ji} \in [0, 1]$ for every edge from upstream component j to downstream component i . If $z_{ji} = 1$, that edge uses the clean activation from the real input. If $z_{ji} = 0$, that edge is replaced by the corrupted activation from a matched counterfactual input. During training, the mask is continuous, so the model can smoothly interpolate between "keep this edge" and "replace this edge." The downstream node i then reads a weighted mix of clean and corrupted inputs and produces its normal output with the original frozen function f_i . The training objective pushes the pruned circuit to match the full model's output while also using as few edges as possible.

Now the full picture: this breaks the usual residual-stream trick. In a normal Transformer, every component writes into one shared running sum, so you can say $h_{i+1} = h_i + y_i$. But here each downstream node may keep some incoming edges and corrupt others. That means different nodes need different versions of the residual stream. One shared sum no longer works. Their fix is the key architectural move: a disentangled residual stream (a residual stream stored as separate past activations instead of one summed vector). Instead of immediately adding outputs together, they keep a list like $(y_0, y_1, \dots, y_{i-1})$. Then, when node i runs, it dynamically builds its own input by summing those earlier activations with the learned edge masks. It is like keeping all ingredients separate until the moment you plate the dish, because each diner wants a different combination.

Training is straightforward once that architecture is in place. They optimize the mask parameters with gradient descent on a loss that includes a faithfulness term and a sparsity term. The faithfulness term is KL divergence (a measure of how different two output distributions are) between the full model and the masked circuit. Lower is better. The sparsity term penalizes keeping too many edges. After optimization, they threshold or discretize the soft masks into binary decisions, giving a final circuit with actual kept and removed edges. At inference time, the circuit is evaluated by corrupting all edges outside the circuit and checking whether the model's behavior stays the same.

The clever bit is that the corruption is not zeroing things out. For interpretability, zero is often the wrong "missing value." If you remove an edge, you want to ask: what would this computation look like if the relevant information came from a corrupted example instead? So they use interchange ablation (replace a signal with the same signal from a corrupted input) rather than blunt deletion. That makes the resulting circuit much more meaningful. In practice they run this on GPT-2 Small with 117M parameters for the main experiments, and even on CodeLlama-13B in a case study. The big message is simple: optimize over edges, not nodes, and make the residual stream flexible enough that each node can see its own custom subgraph.

Prune roads, not buildings

Most prior methods remove whole components like heads or MLPs. That is too coarse if your goal is mechanistic understanding. Edge Pruning works at the level of individual connections, so it can say not just "this head matters" but "this head matters only because it sends this specific signal to that specific place."

Corrupt, don't zero out

A missing edge should not act like silence. It should act like the wrong information. Replacing a removed edge with the corresponding activation from a corrupted example makes the test much sharper: did this exact pathway carry task-relevant information, or not?

Disentangled residual stream

This is the engineering trick that makes the whole method possible. Once different downstream nodes keep different subsets of incoming edges, one shared residual sum is no longer valid. By storing past activations separately and combining them on demand, each node can read exactly the version of history its mask specifies.

Gradient descent beats local guesses

ACDC and EAP rely on local or first-order approximations. Those can work on toy-ish tasks, but they start to crack on richer distributions. Edge Pruning directly optimizes the global objective, which is slower than a cheap heuristic but much more reliable when the task has many templates or more complicated interactions.

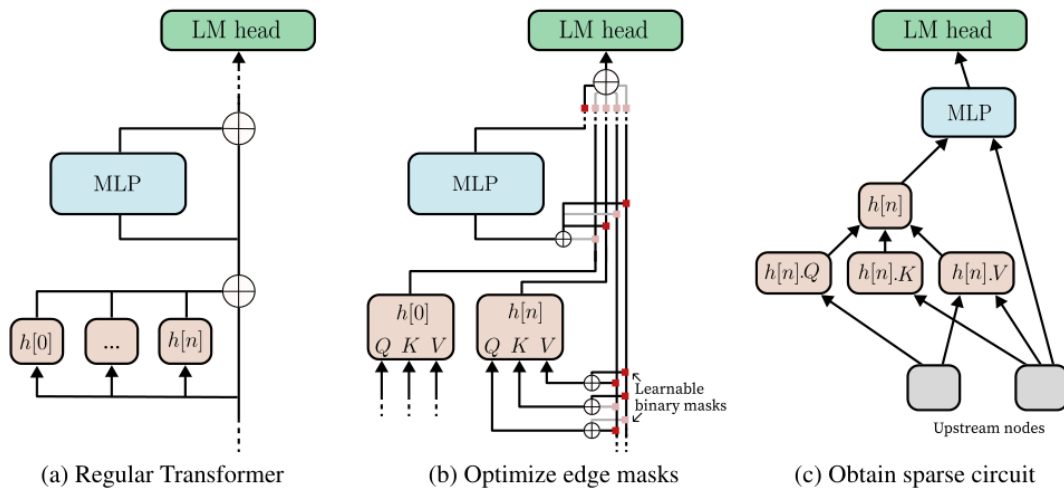


Figure 1: Edge Pruning disentangles the residual stream and optimizes continuous masks on the read operations via gradient descent. Discretizing the masks to $\{0, 1\}$ yields the final circuit. The full model corresponds to the case where all masks equal 1.

This is the whole method in one picture. The important thing to notice is the move from a single residual stream to a disentangled one, plus the per-edge continuous masks. That architectural change is not cosmetic; it is what makes edge-level pruning mathematically coherent.

The Math

Let the Transformer be written as a sequence of components. Component i computes an output $y_i = f_i(\cdot)$ from earlier activations. Here y_0 is the embedding input, and for each edge from component j to component i we introduce a trainable mask $z_{ji} \in [0, 1]$. We also define a corrupted input $\tilde{x}$ and its

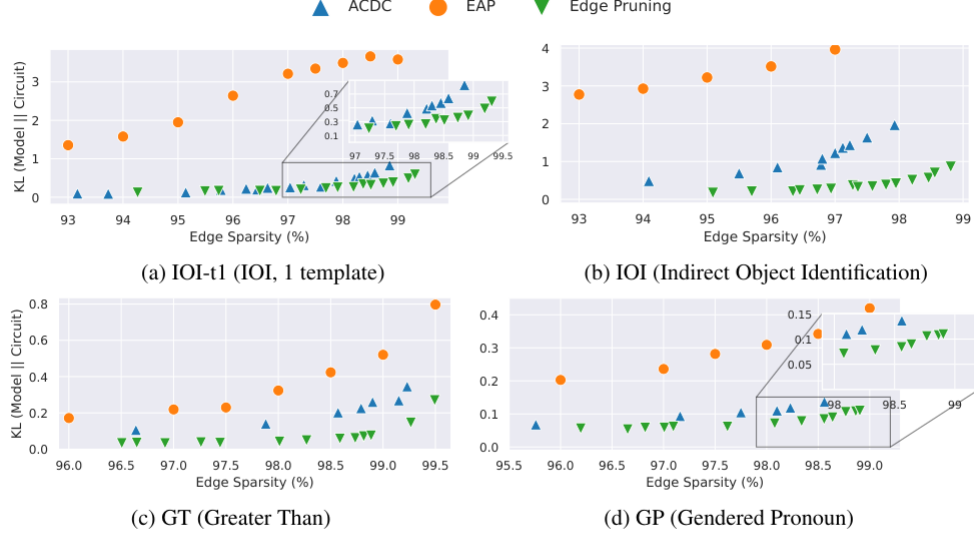


Figure 2: The faithfulness of the methods, given the KL divergence between the model and obtained circuits (*lower is better*). On IOI-t1 and GP, Edge Pruning is competitive at low sparsities and better at high sparsities. It outperforms both ACDC and EAP by a significant margin on IOI and GT.

This is the faithfulness plot, using KL divergence, and it is the fairest scoreboard in the paper. Lower is better. The pattern to watch is that Edge Pruning is only modestly better on the easy tasks, but clearly better on full IOI and Greater Than as sparsity gets extreme.

corresponding activations $\tilde{y}_j$.

The core masked computation is:

$$y_i = f_i \left(z_{0i} y_0 + (1 - z_{0i}) \tilde{y}_0 + \sum_{\substack{1 \leq j < i \\ j \text{ upstream of } i}} (z_{ji} y_j + (1 - z_{ji}) \tilde{y}_j) \right). \quad (1)$$

In plain English: every incoming edge to node i gets its own dial. Turn the dial to 1 and you use the real signal. Turn it to 0 and you swap in the corrupted version of that same signal. The node then runs normally on this mixed input.

In a standard residual-stream Transformer, one would write:

$$h_{i+1} = h_i + y_i. \quad (2)$$

This means every component writes into one shared running total. That is fine when all later nodes read the same history, but it breaks here because different later nodes may keep different subsets of edges.

So they move to a disentangled residual stream. Instead of one summed state, they keep the list of previous activations:

$$\mathcal{H}_i = (y_0, y_1, \dots, y_{i-1}). \quad (3)$$

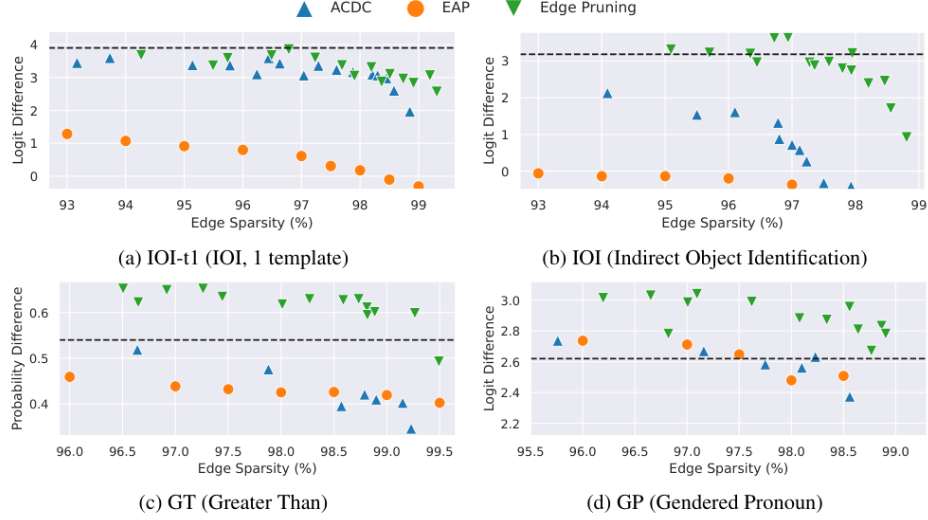


Figure 3: Comparison of circuit performance between methods. We report the Logit Difference $\log P(\text{correct}) - \log P(\text{misleading})$ for IOI-t1, IOI and GP, and the probability difference $P(yy + 1 : 99) - P(00 : yy - 1)$ for GT. Higher is better for all plots. Edge Pruning finds better-performing circuits on all four tasks. The dashed line indicates the performance of the full model.

This figure asks a different question: not just 'does the circuit mimic the model distribution?' but 'does it still do the task well?' Edge Pruning wins on all four tasks. That matters because a circuit can be faithful in KL yet still lose the task margin in ways humans care about.

This means the model stores the ingredients separately instead of pre-mixing them. Then each node can build its own custom input from that list using its own edge masks.

For node i , define its masked aggregated input as:

$$r_i(z) = z_{0i}y_0 + (1 - z_{0i})\tilde{y}_0 + \sum_{\substack{1 \leq j < i \\ j \text{ upstream of } i}} (z_{ji}y_j + (1 - z_{ji})\tilde{y}_j). \quad (4)$$

This is the actual "what information reaches node i ?" equation. Each edge independently chooses between clean and corrupted information, so the circuit can be very precise.

The circuit is trained to match the full model's output distribution. Let $p_{\text{model}}(\cdot | x)$ be the frozen full model distribution on clean input x , and let $p_{\text{circuit}}(\cdot | x; z)$ be the masked circuit distribution. The faithfulness loss is:

$$\mathcal{L}_{\text{KL}}(z) = D_{\text{KL}}(p_{\text{model}}(\cdot | x) \| p_{\text{circuit}}(\cdot | x; z)). \quad (5)$$

This says: make the pruned circuit behave like the original model, not just get the final answer right sometimes. It is a stricter target, because it tries to match the whole output distribution.

To encourage sparsity, they add a penalty on the masks. A generic form is:

$$\mathcal{L}_{\text{sparse}}(z) = \lambda \sum_{(j,i) \in G} z_{ji}, \quad (6)$$

where G is the set of all edges and $\lambda > 0$ controls how hard we push for fewer edges.

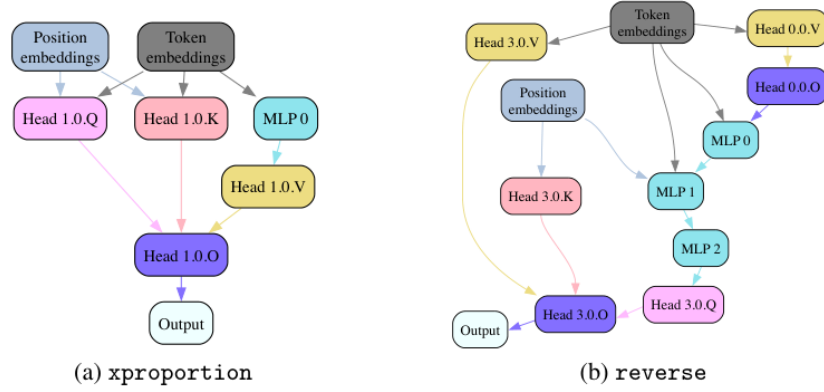


Figure 4: The canonical ground-truth circuits for the Tracr-compiled `xproportion` and `reverse` programs. Edge Pruning recovers both circuits perfectly.

This is the sanity-check figure. On Tracr, the true circuit is known by construction, so there is nowhere to hide. Perfect recovery here is a strong sign that the method can identify real structure when the ground truth exists.

This is just a soft version of "pay a cost for every edge you keep." Larger λ gives smaller circuits, but if you push too hard you start deleting important pathways.

The full optimization problem is then:

$$\min_{z \in [0,1]^{|G|}} \mathcal{L}(z) = \mathcal{L}_{\text{KL}}(z) + \mathcal{L}_{\text{sparse}}(z). \quad (7)$$

This is the whole method in one line. Find the smallest set of edges that still makes the circuit imitate the full model.

Because the masks are continuous, gradients can flow through them. For a single edge mask z_{ji} , the gradient has the form:

$$\frac{\partial \mathcal{L}}{\partial z_{ji}} = \frac{\partial \mathcal{L}_{\text{KL}}}{\partial z_{ji}} + \lambda \quad (8)$$

$$= \left\langle \frac{\partial \mathcal{L}_{\text{KL}}}{\partial r_i}, y_j - \tilde{y}_j \right\rangle + \lambda, \quad (9)$$

where $\langle \cdot, \cdot \rangle$ is the inner product and r_i is the masked input to node i .

This is intuitive once you stare at it. If the clean and corrupted activations are very different, and that difference strongly changes the KL loss, then the edge gets a strong learning signal. If keeping the clean signal helps match the model, the optimizer pushes z_{ji} upward.

After training, the soft masks are discretized into a binary circuit. A simple thresholding rule is:

$$\hat{z}_{ji} = \begin{cases} 1, & z_{ji} \geq \tau, \\ 0, & z_{ji} < \tau, \end{cases} \quad (10)$$

where τ is a threshold chosen to hit a desired sparsity level.

This turns the fuzzy "maybe keep this edge" solution into an actual graph you can inspect. The final circuit is then evaluated by corrupting all edges with $\hat{z}_{ji} = 0$ and leaving the kept edges untouched.

For the task metrics, they report logit difference on IOI and GP:

$$\text{LogitDiff} = \log P(\text{correct}) - \log P(\text{misleading}). \quad (11)$$

This is a clean way to ask whether the model prefers the right answer over the tempting wrong one. Bigger is better.

For Greater Than, they use probability difference over the correct and incorrect year ranges:

$$\text{ProbDiff} = P(y_y + 1 : 99) - P(00 : y_y - 1). \quad (12)$$

This checks whether the model puts more mass on years greater than the given prefix target than on smaller years. Again, bigger is better.

And their main faithfulness metric is KL divergence between the full model and the circuit:

$$D_{\text{KL}}(p||q) = \sum_{v \in \mathcal{V}} p(v) \log \frac{p(v)}{q(v)}, \quad (13)$$

where $\mathcal{V}$ is the output vocabulary, p is the full model distribution, and q is the circuit distribution.

This is the right metric if you care about preserving behavior, not just preserving top-1 accuracy. Two models can give the same answer but for very different reasons; KL catches that difference.

Does It Work?

The experimental setup is stronger than a lot of circuit papers because they actually use train, validation, and test splits instead of evaluating on the same examples used to find the circuit. That matters. On GPT-2 Small (117M), they test four tasks: IOI-t1 with 50 examples per split, full IOI with 200 train, 200 validation, and 36,084 test examples across 30 templates, Greater Than with 150 train, 150 validation, and 12,240 test examples, and Gendered Pronoun with 150 train, 150 validation, and 378 test examples. They compare against ACDC and EAP, which are the right baselines here.

The headline result is simple: Edge Pruning wins clearly on the harder tasks. On Figure 2, it is only roughly competitive on IOI-t1 and GP at low sparsity, then pulls ahead at higher sparsity. But on full IOI and Greater Than, it beats both ACDC and EAP by a wide margin in KL faithfulness. That is impressive because those are exactly the settings where local approximations should start to fail. The paper even points out a brutal detail: on IOI at high sparsity, ACDC gets KL around 0.69, which is basically as bad as random choice between the two names. Edge Pruning still holds up. In Figure 3, it also finds better-performing circuits on all four tasks. The nicest concrete comparison: on IOI, Edge Pruning gets a circuit at 98.8% sparsity that matches the faithfulness and performance of ACDC's 96.8% sparsity circuit. That means over 2.65× fewer edges for the same quality. That is exactly the kind of result you want from a circuit-finding method.

The scaling table is maybe the most convincing part. On the larger IOI dataset, all methods are tested at 96.6% sparsity. With 200 examples, ACDC gets KL 0.92 in 18,783 seconds, EAP gets 3.47 in 21 seconds, and Edge Pruning gets 0.25 in 2,756 seconds. With 400 examples, ACDC improves to 0.88 but runtime

explodes to 40,759 seconds; EAP stays fast but weak at 3.66 in 43 seconds; Edge Pruning improves to 0.22 in 2,931 seconds. At 100K examples, ACDC is not even reported, EAP gets 3.78 in 12,260 seconds, and Edge Pruning gets the best KL by far, 0.20, in 3,042 seconds. That is a strong practical story: not just better circuits, but better scaling when you actually need lots of data.

The Tracr result is clean and satisfying: perfect recovery of the ground-truth circuits for xproportion and reverse. The 13B case study is also striking. On CodeLlama-13B, they find circuits with just 1,041 edges for instruction prompting and 1,464 edges for few-shot prompting, which is 99.97% and 99.96% sparsity. Those tiny circuits still get 79.25% and 87.25% accuracy, versus the full model’s 82.00% and 89.25%. That is impressive because the full model has 3,872,820 edges in their graph view. But notice the gap in evaluation: this large-model section is a case study, not a broad benchmark. They do not compare against ACDC or EAP there, presumably because those methods do not scale, but it still means the large-scale claim is less controlled than the GPT-2 experiments.

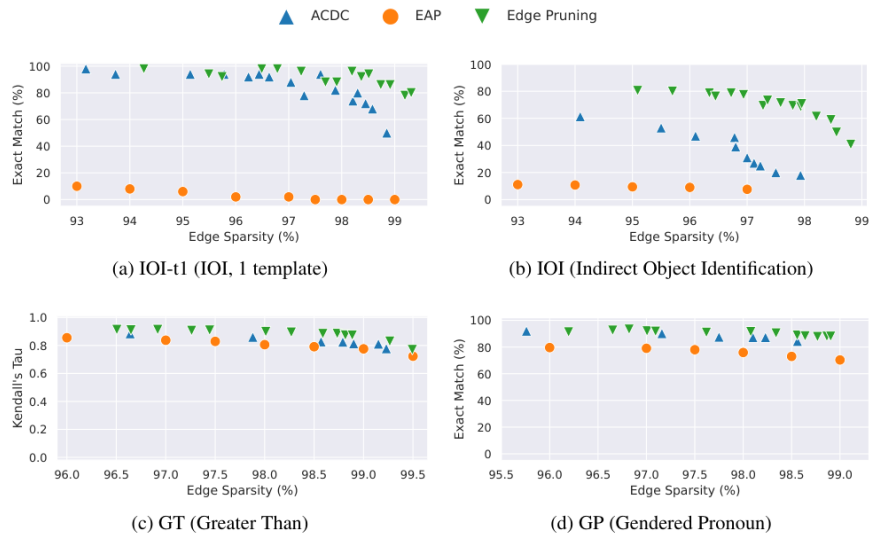


Figure 5: Our secondary metric for measuring faithfulness is the Exact Match percentage between the model and circuit predictions on IOI-t1, IOI, and GP. On GT, we use the Kendall’s Tau score between the model and circuit rankings of 00, 01, . . . , 99 as the secondary metric. Edge Pruning is the most faithful method on all four tasks, with the difference being especially pronounced for IOI.

Figure 5: Our secondary metric for measuring faithfulness is the Exact Match percentage between the model and circuit predictions on IOI-t1, IOI, and GP. On GT, we use the Kendall’s Tau score between the model and circuit rankings of 00, 01, . . . , 99 as the secondary metric. Edge Pruning is the most faithful method on all four tasks, with the difference being especially pronounced for IOI.

Blind Spots & Gotchas

Blind Spots & Gotchas

- Implementation gotcha: if you keep the standard shared residual stream instead of the disentangled residual stream, the method is wrong. Different downstream nodes need different masked combinations of earlier activations. One shared sum silently mixes incompatible worlds and gives meaningless gradients.
- Implementation gotcha: the corrupted activations must come from aligned counterfactual examples. If you swap in arbitrary unrelated examples, the ablation target becomes noisy and the learned circuit can chase artifacts instead of real task structure.
- Implementation gotcha: discretization matters. A soft mask solution can look great during optimization, then collapse when thresholded to binary edges. If you do not validate the threshold or target sparsity carefully, you can report a circuit that only worked in its fuzzy training form.
- Paper blind spot: they mostly evaluate on tasks with fairly synthetic structure: IOI, Greater Than, Gendered Pronoun, Tracr, and Boolean expressions. These are useful, but they are not messy real-world language understanding. We still do not know how well edge-level circuits behave on open-ended tasks with many valid reasoning paths.
- Paper blind spot: faithfulness is measured relative to a chosen corruption scheme. If interchange ablation is the wrong notion of 'remove this information' for some task, the recovered circuit may be faithful to the metric but not to the true causal mechanism.

So What?

This paper matters because it upgrades circuit finding from a rough sketch to something much closer to a wiring diagram. Before this, a lot of methods could tell you which components seemed important. Useful, but blurry. Edge Pruning says which exact pathways carry the computation. That is a big deal for mechanistic interpretability, because mechanisms live in interactions, not just in isolated heads or MLPs.

In practice, this gives you a tool for extracting sparse, testable subgraphs from large models. That is useful far beyond toy interpretability demos. In quantitative finance, you could use the same idea to isolate the exact pathways a Transformer uses for earnings-call sentiment, guidance extraction, macro surprise interpretation, or order-book forecasting. For portfolio optimization and risk, you could compare circuits across market regimes and ask whether the model uses the same mechanism in calm periods and crisis periods. For execution, you could test whether a model's short-horizon trading decisions rely on stable microstructure signals or brittle prompt artifacts. For NLP on filings, you could isolate the edges responsible for reading footnotes, debt covenants, or management guidance and see whether those pathways transfer across sectors.

A concrete extension: do edge pruning across time or regime slices and compare circuit overlap. For example, train one circuit on pre-2020 market data and one on post-2020 data, then measure which edges survive. That would tell you whether the model learned a stable mechanism or just a period-specific shortcut. Closing thought: the paper's core contribution is not just a better pruning trick. It is a better question. Not "which parts matter?" but "which connections actually carry the thought?"